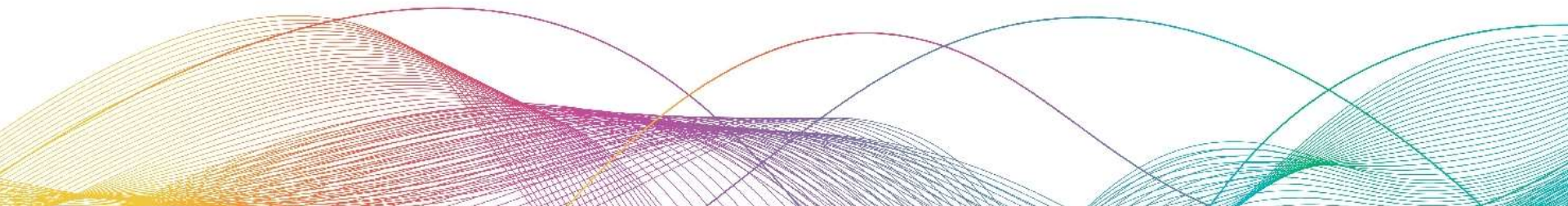


05

Build A Complex Application

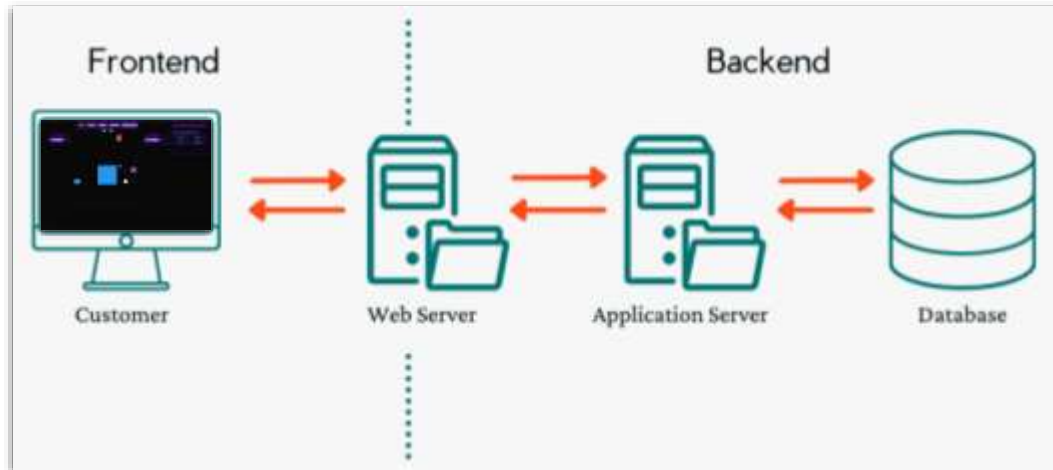


Introduction to Web Applications

A web application is made up of two main parts:

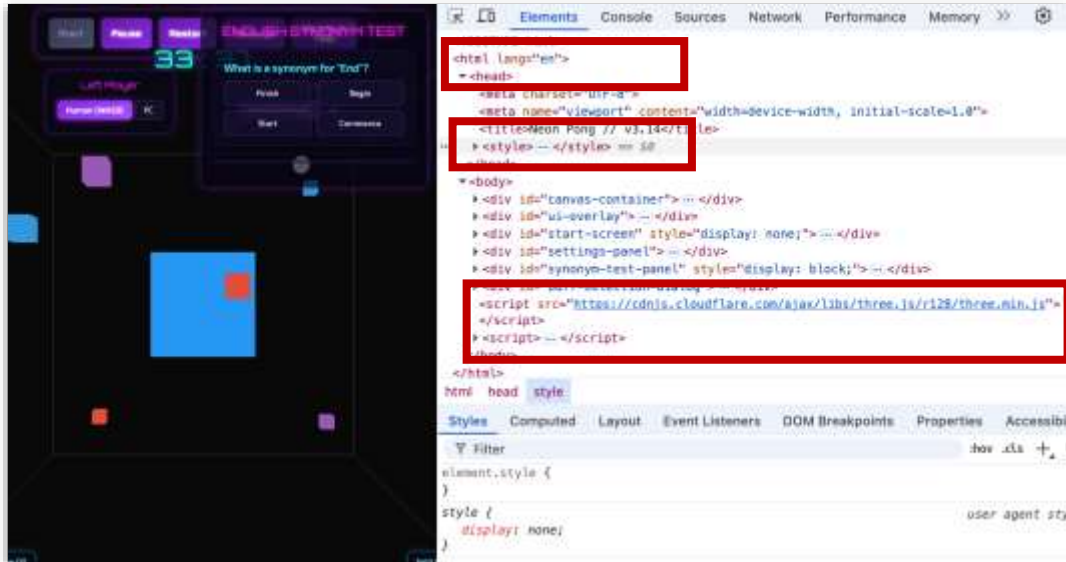
- **Frontend:** what users see and interact with
- **Backend:** the server that processes data and logic

Together, they create dynamic, interactive websites



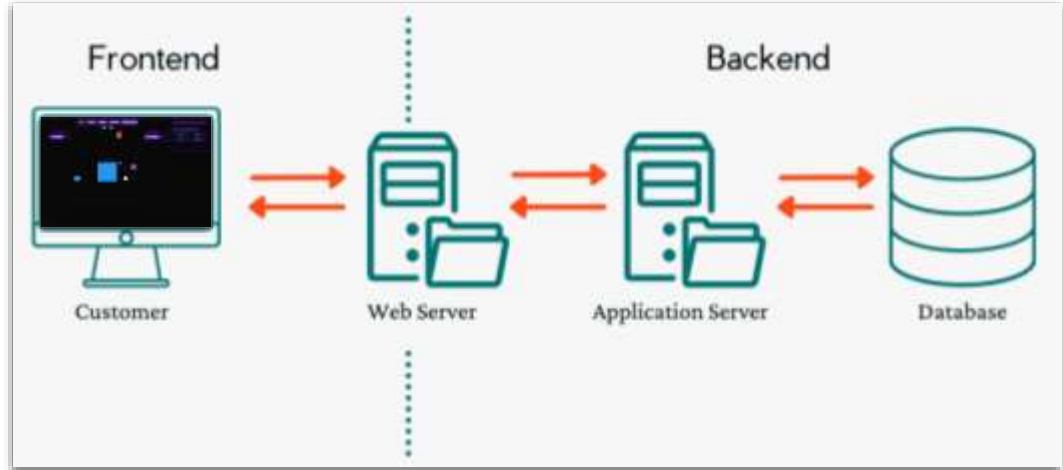
The Frontend

- Runs in the **browser**
- Built with:
 - HTML → structure
 - CSS → style
 - JavaScript → behavior
- Example: displaying a quiz question to the user



The Backend

- Runs on the **server**
- Handles:
 - Data storage (databases)
 - Logic and computation
 - Security and validation
- Communicates with frontend through HTTP requests
 - JSON or HTML responses



When Do We Need a Backend?

- When we need to **store data** or **process user input**
- When we need **secure checks** or **business logic**
- Example :
 - A test question appears in the frontend
 - The user submits an answer
 - The backend checks if the answer is correct and returns a result

1. Frontend shows a question, e.g. "What is 2 + 2?"
2. User enters an answer, e.g. "5" and clicks "Submit"
3. Frontend sends "{answer: 5}" to backend
4. Backend returns result → "Incorrect!"
5. Frontend displays the result

Popular Python Backend Frameworks

- **Flask**
 - Lightweight and flexible
 - Great for small to medium apps
- **Django**
 - Full-featured and opinionated
 - Includes admin, authentication

Both can serve HTML templates and static files



Preparation for Flask

- Open the command line window
 - macOS: Terminal
 - Windows: Anaconda Prompt

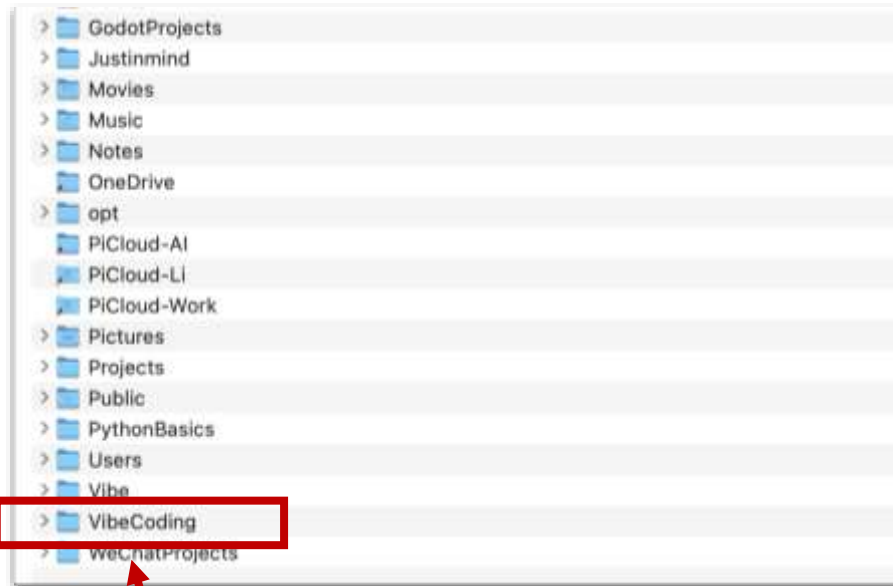
`cd /Users/[name]`

`cd C:\Users\[name]`

```
mkdir VibeCoding
```

```
cd VibeCoding
```

```
pip install flask
```



You should see the created folder in the home directory.

How Flask Works

- Flask maps **URLs** to **Python functions**
- These functions return **HTML** or **JSON** responses

Open Terminal (Terminal for macOS, Anaconda Prompt for Windows)

```
python hello.py
```

Open “localhost:6660” in browser

```
# hello.py

from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Hello, World!"

if __name__ == '__main__':
    app.run(debug=True, port=6660)
```


DB-Demo

Prompt:

1. A simple web application demonstrating database CRUD (Create, Read, Update, Delete) operations
2. HTML+Flask+DB (JSON)
3. The interface is nice.

 **DB Demo**
Create · Read · Update · Delete

[View JSON](#)

 [Add New Record](#)

Name:

Email:

Phone:

[Add](#)

 [Data List](#)

ID	Name	Email	Phone	Actions
1	Alice Chen	alice@example.com	13800138001	Edit Delete
2	Bob Wang	bob@example.com	13800138002	Edit Delete
3	Charlie Li	charlie@example.com	13800138003	Edit Delete
4	David Zhang	48200213@qq.com	15610099133	Edit Delete
5	Emma Liu	emma@example.com	13810099134	Edit Delete
6	Faye	Faye@123.com	123456789	Edit Delete

Plane-Shooter

Using HTML+Flask+DB (JSON) technology stack, create a 2D vertical shooting game or other games or learning applications.

Requirements:

1. Other students can access it through the internet and start using it by entering their names.
2. During use, there are points and a ranking list, sorted according to the high or low points.

```
PS C:\Users\FQM\Desktop\workspace> & c:/Users/FQM/Desktop/workspace/.ane-shooter/app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.246:5000
Press CTRL+C to quit
```

Other students can access it by entering this address in their browser

Plane-Shooter



Plane Shooter

Enter your username to start playing!

Logged out successfully! 🎮

 Username

FQM

 Start Playing

No password needed - just enter your username!

 View Leaderboard |  Game Rules



Leaderboard

Top 10 Pilots

RANK	PILOT	SCORE	LEVEL	DATE
	 maximuz	24100	★ 5	2026-03-11 16:57:47
	 Ace	21000	★ 6	2024-01-14 18:45:00
	 Star	18000	★ 6	2024-01-12 14:20:00
#4	Player1	15000	★ 6	2024-01-10 15:30:00
#5	Ace	12500	★ 5	2024-01-11 16:00:00
#6	maximuz	10800	★ 3	2026-03-11 16:52:20
#7	Player1	9500	★ 4	2024-01-13 10:00:00
#8	fqm	6950	★ 2	2026-05-15 20:38:41
#9	maximuz	6850	★ 6	2026-03-11 16:41:39
#10	maximuz	900	★ 1	2026-03-11 17:14:40

[← Back to Login](#) [Game Rules](#)

Automated Testing

Steps:

1. Write a test checklist using AI.
2. Use AI to install testing environment dependencies (When first used).
3. Create test file structure using AI.
4. Conduct API testing
5. Conduct UI testing
6. Run all tests (some tests require manual completion)
7. View test results

测试清单

阶段一

- [] 单人模式正常运行
- [] 双人模式正常运行
- [] 波次系统正确推进
- [] BOSS 在所有波次结束后出现
- [] 击败 BOSS 触发通关
- [] 道具正确生成和生效
- [] 生命系统正常运作
- [] 分数计算正确
- [] 6 个关卡均可玩
- [] 游戏结束和重新开始正常

阶段二

- [] 音效正常播放
- [] 背景音乐正常播放
- [] 音效/音乐开关有效
- [] 难度设置影响游戏平衡
- [] 生命数设置生效

阶段三

- [] 用户登录/注册正常
- [] 分数正确保存和读取

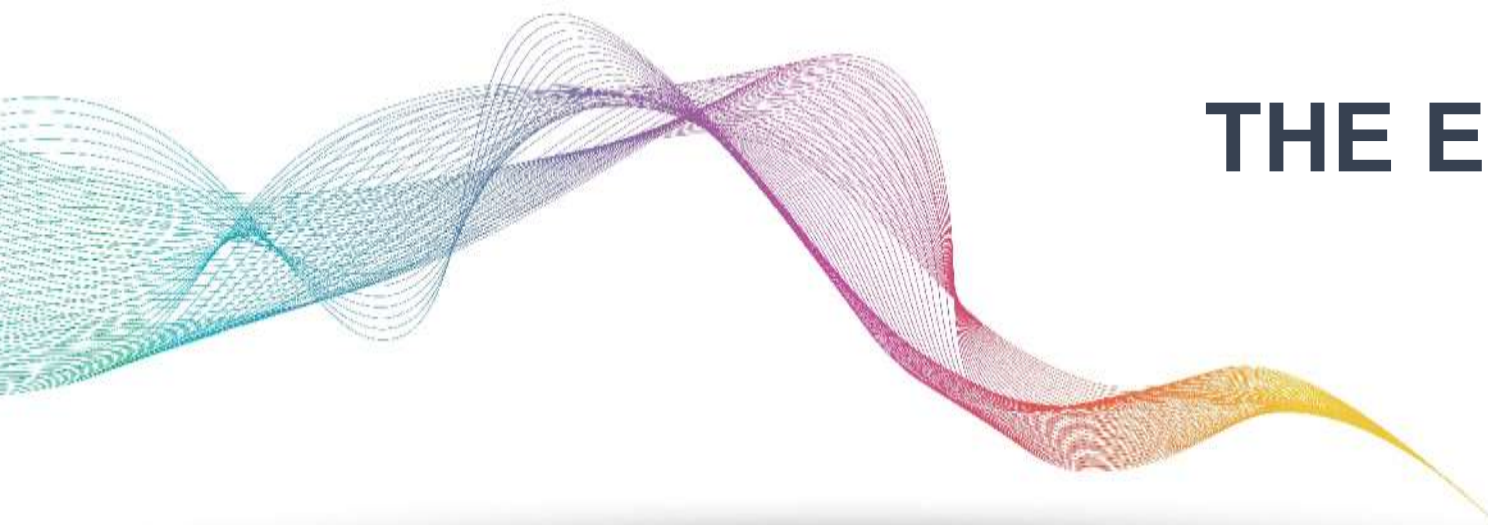
现在安装测试依赖并运行测试。

命令已执行 `cd c:\Users\FQM\Desktop`

需要先安装 Flask 依赖。让我安装项目所需的依赖。

命令已执行 `cd c:\Users\FQM\Desktop\workspace`

阶段	测试清单	状态
阶段一	单人/双人模式、波次系统、BOSS 出现、道具生成、生命系统、分数计算、关卡初始、游戏结束	✅ API 测试完成
阶段二	音效/音乐播放、开关控制、难度设置、生命数设置	🔴 需手动测试
阶段三	用户登录/注册、分数保存、排行榜显示、导出功能	✅ API 测试完成



THE END